

경구약제 이미지 객체 검출 프로젝트

목차

[경구약제 이미지 객체 검출 프로젝트](#)

[도입 배경](#)

[Problem: 육안 기반 경구 약제 검수의 한계와 조제 오류 위험](#)

[Solution: AI 기반 객체 탐지\(Object Detection\)를 통한 검수 자동화](#)

[목적](#)

[목표](#)

[1. 정량적 목표 \(Quantitative Goals\)](#)

[2. 정성적 목표 \(Qualitative Goals\)](#)

[역할 분담](#)

[👥 팀 1423](#)

[일정](#)

[공동 개발 환경](#)

[아키텍처](#)

[데이터셋](#)

[경구 약제 데이터 특징](#)

[데이터셋 구조](#)

[데이터셋 분할](#)

[데이터 탐색 및 전처리](#)

[데이터 구조 문제 및 병합 전략 \(전처리\)](#)

[file_name 기준 JSON 그룹핑 및 병합](#)

[bbox 존재 및 유효성 필터링](#)

[image_id / category_id / annotation_id 재부여](#)

[데이터 탐색\(EDA\) 및 인사이트](#)

[기타](#)

[사용 모델](#)

[YOLO \(Ultralytics 계열\)](#)

[실험결과\(Base\)](#)

[성능/정확도 평가](#)

[\[Yolo\] 정성적 평가](#)

[\[Yolo\] 정량적 평가](#)

[실험결과\(Base 개선\)](#)

[1. YOLO11의 성능이 높았던 이유](#)

[2. s 모델의 성능이 상대적으로 낮았던 이유](#)

[3. YOLO26의 성능이 낮았던 이유](#)

[4. 결론](#)

[실험결과\(AI HUB 데이터 확보\)](#)

[실험결과\(yolo11n vs yolo11l\)](#)

[실험결과\(yolo vs Faster R-CNN\)](#)

[결론](#)

경구약제 이미지 객체 검출 프로젝트

도입 배경

Problem: 육안 기반 경구 약제 검수의 한계와 조제 오류 위험

- **조제 업무의 피로도 및 인적 오류(Human Error):** 경구 약제(알약)는 미세한 각인, 식별 문자, 유사한 형태/색상으로 인해 육안 판별 시 검수 시간이 길어지고 판별 오류 위험이 상존합니다.
- **안전과의 직결성:** 헬스케어 및 의약 분야의 오투약 사고는 환자의 안전과 직결되므로, 미세한 식별 오류도 허용되지 않는 **고정밀(High Precision)** 관리가 필수적입니다.

Solution: AI 기반 객체 탐지(Object Detection)를 통한 검수 자동화

- **고정밀 컴퓨터 비전 기술 적용:** 알약의 미세 각인 및 외형 특징을 인식하는 AI Detection 모델을 구축하여 식별 정확도를 극대화합니다.
- **검수 공정 효율화:** 수작업에 의존하던 약제 검수 프로세스를 자동화하여 조제 오류율(Medication Error Rate)을 획기적으로 낮추고 작업 생산성을 제고합니다.

목적

- **최적의 경구 약제 Detection 모델 선정을 위한 비교 연구**
 - 헬스케어 도메인 특성상 요구되는 고정밀 검출(High Precision)과 최적의 밸런스를 갖춘 객체 검출 모델을 탐색함
- **서로 다른 메커니즘을 가진 2종의 대표 Detection 모델 성능 평가**
 - **1-Stage Detector (YOLO / Ultralytics 계열):** 영역 제안과 분류를 동시에 수행하여, **추론 효율성**을 검증
 - **2-Stage Detector (Faster R-CNN):** 객체 영역 제안(Region Proposal)과 분류를 분리 수행하여, 소형 알약 및 미세 각인에 대한 **정밀도(Accuracy/mAP) 극대화** 가능성을 검증
- **정량적 지표 기반의 객관적 최종 모델 확정**
 - 동일한 경구 약제 dataset을 활용하여 두 모델의 **mAP(Mean Average Precision)** 등을 다각도로 비교 분석하고, 가장 적합한 모델을 최종 선정함

목표

동일한 경구 약제 데이터셋을 활용하여 두 모델(YOLO, Faster R-CNN)의 성능을 다각도로 비교 분석하고, 가장 적합한 모델을 최종 선정하는 것을 목표로 합니다. 이를 위해 정량적 지표 기반의 객관적 기준과 정성적 검증 기준을 다음과 같이 수립했습니다.

1. 정량적 목표 (Quantitative Goals)

- **종합 객체 탐지 성능:** Kaggle 리더보드 평가 기준을 반영하여 전체 56개 클래스에 대한 평가점수 mAP(평균평균정밀도) [0.75:0.95] 40% 이상 달성
- **안전성 최우선 지표 확보 (미탐지 제로화):**
 - 오투약 사고를 원천 방지하기 위해 알약 누락(미탐)을 최소화하는 **Recall(재현율) 99% 이상** 달성

2. 정성적 목표 (Qualitative Goals)

- **최적의 아키텍처 검증:** 1-Stage Detector(YOLO 계열), 2-Stage Detector(Faster R-CNN)의 식별 능력과 추론 효율성을 비교하여 최적의 밸런스를 갖춘 모델 확정
- **극한 환경(Edge Case) 강건성 분석:** 조명 변화(빛 반사, 그림자) 및 다수의 알약이 있는 상황(Occlusion)에 서도 바운딩 박스를 안정적으로 분리하는지 시각적 검증 수행

역할 분담

👥 팀 1423

역할명	담당자	상세
Project Manager	서동현	- 프로젝트의 협업 과정을 매니징하는 역할 - 애자일/스프린트 방식으로 단위를 분리하고 회의를 주도
Data Engineer	김주희	- 데이터 수집, 정제 및 전처리를 담당하고 데이터 파이프라인을 구축
Model Architect	조영권	- 딥러닝 모델을 설계하고 아키텍처를 결정
Experimentation Lead	김원태, 이석우	- 다양한 실험을 주도하고, 하이퍼파라미터 튜닝 및 모델 성능 평가를 담당

일정

프로젝트 WBS (Work Breakdown Structure)							
No	구분(Phase)	세부 업무	담당자	시작일	종료일	기간(일)	비고
1	초기 셋팅	Github Repository 생성 및 초대	서동현	2026-09-11	2026-09-14	4	주말 2일 제외 필요
2		공통 개발 환경 셋팅	서동현	2026-09-11	2026-09-14	4	주말 2일 제외 필요
3		Kaggle 팀 생성 및 초대	서동현	2026-09-11	2026-09-14	4	주말 2일 제외 필요
4	EDA, 전처리	EDA, 전처리	김주희 서동현	2026-09-11	2026-09-14	4	주말 2일 제외 필요
5	모델 및 실험	모델 학습 및 실험	김원태 이석우 조영권	2026-09-15	2026-09-21	7	주말 2일 제외 필요
6		↳ Matplotlib 등으로 데이터 시각화 및 모델 성능 결과 도출		2026-09-15	2026-09-21	7	Kaggle Leaderboard 제출 Score 확인. 개인 제출 X, 팀당 하루 10번 제한
7	보고서 및 발표자료 작성	보고서 및 발표자료 작성	전 인원	2026-09-22	2026-09-23	2	각 담당 업무 관련 작성
8		↳ 보고서 및 발표자료 최종 보완		2026-09-28	2026-09-28	1	비연속 일정 (09.22~23, 09.28)
9	제출	결과물 제출	서동현	2026-09-29	2026-09-29	1	마감 19:00까지
10	발표 및 협업일지	발표	전 인원	2026-09-30	2026-09-30	1	
11		↳ 협업일지 작성 및 제출		2026-09-30	2026-09-30	1	마감 23:50까지

공통 개발 환경

1. 개발 환경 세팅 및 형상 관리 전략

본 프로젝트는 팀원 간의 원활한 협업과 결과물의 완벽한 재현성(Reproducibility)을 보장하기 위해 통합된 가상 환경을 구축하여 사용했습니다. 개별적인 패키지 설치로 인한 의존성 충돌(Dependency Hell)을 방지하고자 `environment.yaml` 을 작성하여 GitHub를 통해 배포 및 동기화 했습니다.

2. 환경 개요 및 재현 가이드

- **Target OS:** Windows
- **Python Version:** 3.14.7
- **패키지 관리:** Conda (주요 환경) 및 Pip (Conda 미지원 패키지) 혼용
- **환경 구성 명령어:** Bash

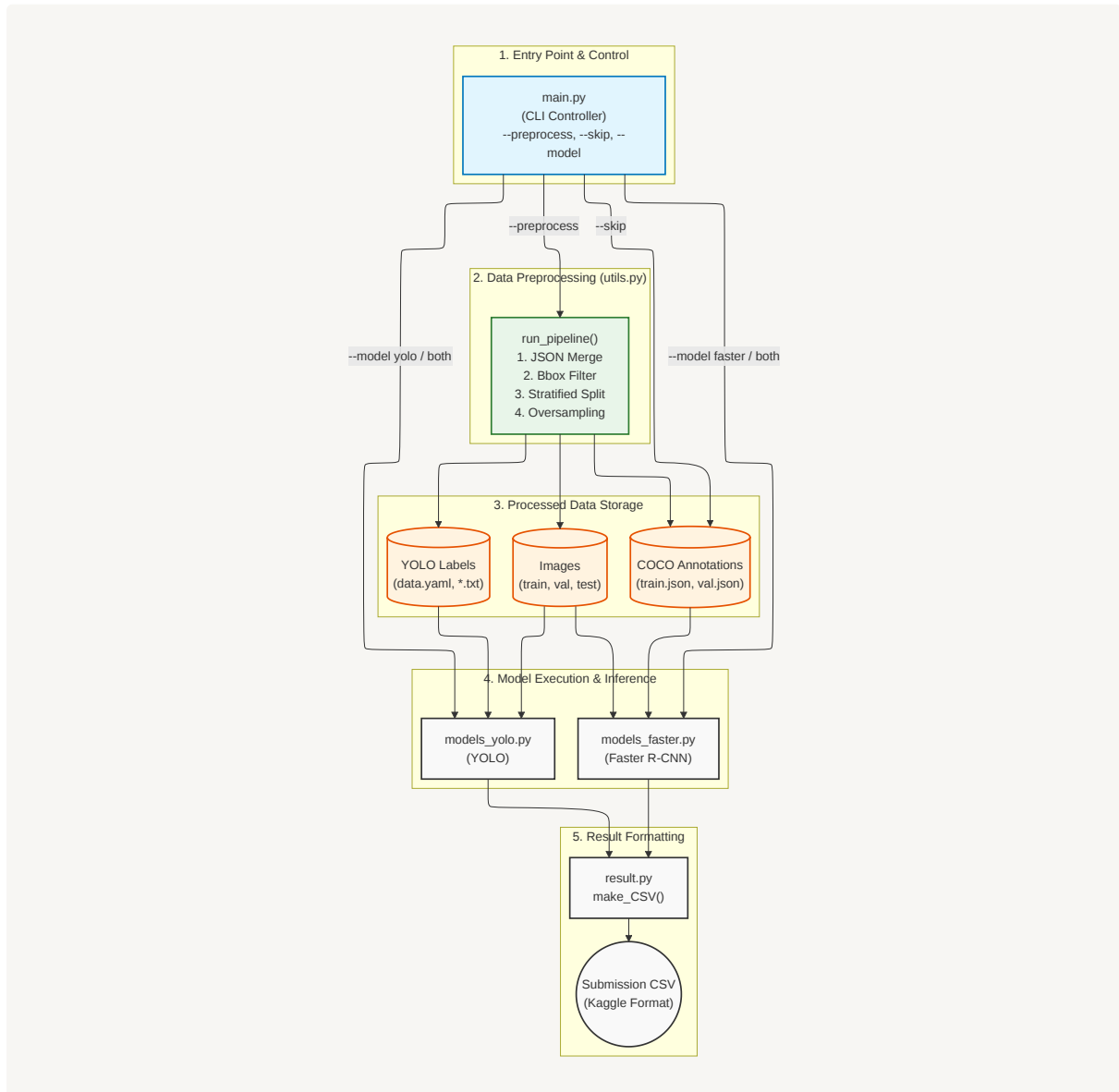
```
conda env create -f environment.yaml
conda activate myenv
```

3. 도메인별 핵심 라이브러리 및 도입 배경

수십 개의 의존성 패키지 중, 본 프로젝트의 핵심 파이프라인 구성을 위해 전략적으로 채택한 주요 라이브러리는 다음과 같습니다.

- **비전 처리 및 전처리 (Computer Vision & Data Processing)**
 - `opencv-python (5.0.0.93)` / `pillow (12.3.0)` : 다중 객체 이미지 로드, 이미지 리사이징 및 바운딩 박스 (Bbox) 좌표 시각화를 위해 도입.
 - `pandas (3.0.5)` / `numpy (2.5.2)` : JSON 어노테이션 파일의 병합, 결측치 필터링 및 다차원 배열 변환 처리.
- **머신러닝 검증 (Machine Learning Validation)**
 - `iterative-stratification (0.1.9)` : 본 프로젝트 데이터의 핵심 난제인 '다중 객체 및 심각한 클래스 불균형' 문제를 해결하기 위해 도입. 다중 레이블 데이터의 계층적 교차 검증(Stratified K-Fold)을 수행하여 모델 학습의 신뢰성을 확보함.
 - `scikit-learn (1.9.1)` / `joblib (1.6.0)` : 데이터 분할 및 학습된 모델의 저장, 병렬 처리 용도.

아키텍처



[아키텍처 설계 의도]

본 프로젝트의 실행 아키텍처는 **단일 진입점(main.py)**을 통한 **제어**와 **기능별 모듈화(전처리, 모델링, 결과 출력)**를 핵심으로 설계되었습니다.

CLI 파라미터를 통해 데이터 전처리 파이프라인(utils.py)의 강제 재실행 또는 스킵을 유연하게 제어할 수 있으며, 전처리가 완료된 공통 산출물(data/)을 바탕으로 `models_faster.py`와 `models_yolo.py`가 상호 간섭 없이 독립적으로 동작합니다. 최종 추론 결과는 `result.py`를 통해 캐글(Kaggle) 제출 양식으로 자동 규격화되어 통합 파이프라인의 효율성을 극대화했습니다.

데이터셋

경구 약제 데이터 특징

- 이미지 및 클래스 구성:
 - 학습용 이미지는 총 232장
 - 검출해야 할 전체 고유 클래스(약 종류)는 56개
- 다중 객체 이미지:
 - 원본 이미지 해상도 976x1280 2~4종류의 알약이 함께 촬영된 조합 이미지 형태를 띠고 있습니다.
 - 병합 전 전체 어노테이션 수는 763개이며, 이미지 1장당 평균 약 3.3개의 알약 객체가 존재합니다.
- 데이터 불균형 (Class Imbalance):
 - 특정 클래스(예: 일양하이트린정 2mg, 153개)에 객체가 집중되어 있는 반면, 하위 5개 클래스는 각각 3번씩만 등장하여 클래스 간 빈도 불균형이 큼.

데이터셋 구조

- 훈련 데이터 (`train_images`): 총 232개의 이미지 파일
- 테스트 데이터 (`test_images`): 총 842개의 이미지 파일
- 훈련 어노테이션 (`train_annotations`): 총 763개의 JSON 파일

데이터셋 분할

1. 분할 전략 개요

- a. 분할 비율: Train 80% / Validation 20%
- b. 핵심 알고리즘: `iterative-stratification` 패키지의 `MultilabelStratifiedShuffleSplit` 을 활용하여 각 클래스의 비율이 훈련셋과 검증셋에 골고루 반영되도록 설계했습니다.

2. 주요 세부 전략

- a. 다중 레이블 계층화 분할 (Multilabel Stratified Split)
 - i. 배경: 알약 검출 데이터셋은 한 이미지에 여러 알약(클래스)이 함께 등장하므로 단순 무작위 분할 시 특정 클래스가 한쪽으로 편중될 위험이 큼.
 - ii. 적용: 모든 이미지와 카테고리 ID를 매핑한 바이너리 형태의 매트릭스를 구성한 뒤, 각 클래스의 출현 빈도를 유지한 채 데이터가 균등하게 나누어지도록 분할을 수행했습니다.
- b. 희귀 클래스 강제 배정 보정 로직 (Rare Class Enforcement)
 - i. 배경: 데이터 총량이 적거나 특정 샘플에만 등장하는 소수 클래스의 경우, 계층화 분할을 거치더라도 검증셋(Validation)에 아예 포함되지 않는 누락 현상(`missing_in_val`)이 발생할 수 있습니다. 이는 모델의 객관적인 검증을 방해합니다.
 - ii. 적용: 분할 완료 후 검증셋에 존재하지 않는 카테고리가 발견될 경우, 해당 클래스를 포함하는 이미지를 추적하여 검증셋에 최소 1장 이상이 강제로 포함되도록 보정하는 후처리 로직을 추가했습니다. 이를 통해 모델이 모든 클래스에 대해 평가될 수 있도록 보장합니다.

iii. 중복 제거 및 무결성 검증 (Integrity Check)

1. 강제 배정 과정에서 발생할 수 있는 파일 중복 문제를 방지하기 위해 파일의 순서 보존 딕셔너리 기법(`dict.fromkeys`)을 활용해 고유 파일 리스트를 재정리했습니다.
2. 최종 단계에서 `missing_train_final` 과 `missing_val_final` 을 출력하여 학습셋과 검증셋 양쪽에 누락된 클래스가 단 하나도 존재하지 않음을 최종 검증합니다.

데이터 탐색 및 전처리

데이터 구조 문제 및 병합 전략 (전처리)

file_name 기준 JSON 그룹핑 및 병합

- **문제** : 원본 데이터는 한 이미지 내에 여러 종류의 알약이 있음에도 불구하고, 각각의 약 종류마다 별도의 JSON 파일이 나뉘어 생성되어 있었습니다 (이미지 232장 대비 JSON 763개).
- **해결 및 이유** : 객체 검출(Object Detection) 모델을 정상적으로 학습시키려면 1장의 이미지에 포함된 모든 바운딩 박스(bbox)가 하나의 파일 안에 통합되어 있어야 합니다. 따라서 동일한 `file_name` 을 가진 여러 JSON의 어노테이션들을 하나로 모아 다중 객체 레이블 구조로 재구성했습니다.

bbox 존재 및 유효성 필터링

- **문제**: 데이터 수집 및 가공 과정에서 `bbox` 키가 누락되거나 비어있는 등 불완전한 어노테이션이 존재할 가능성이 있었습니다.
- **해결 및 이유** : `bbox` 키가 존재하고, 그 값이 `[x, y, width, height]` 형태의 길이 4인 리스트 조건을 만족하는 유효한 데이터만 필터링 했습니다. 이는 잘못된 좌표 형식으로 인해 발생하는 모델 학습 에러를 사전에 차단하기 위함입니다.

image_id / category_id / annotation_id 재부여

- **문제**: 원본 파일들에서 `image_id` , `category_id` , `annotation_id` 등의 식별 값이 모두 1로 일괄 설정되어 있어, 데이터 간의 고유 식별이 불가능하고 병합 시 ID 중복 충돌이 발생했습니다.
- **해결 및 이유**: 각 요소가 COCO 포맷 내에서 충돌 없이 참조될 수 있도록 고유 번호를 재할당 했습니다. 이미지는 파일명을 기준으로 순차 부여하고, 어노테이션은 유효성 필터링 후 고유 ID를 부여했으며, 클래스는 약품의 고유 코드인 `dl_idx` 값을 정수형으로 할당하여 정보의 무결성을 확보했습니다.

데이터 탐색(EDA) 및 인사이트

- **클래스 불균형(Class Imbalance) 발견**: 총 56개의 클래스 분포를 확인한 결과, 최다 빈도 약제는 153번 등장하지만 17개의 약제는 단 3번만 등장하는 극심한 불균형을 확인했음.
- **데이터 조합의 특징 파악**: 어노테이션 수(`count`)와 이미지 수(`num_images`)가 완전히 동일한 것을 확인하여, 한 이미지 내에서 동일한 약이 중복으로 등장하지 않는다는 중요한 규칙을 찾아냄.

기타

1. 데이터 증강 시 유의할 점

확인 사항	증강에 주는 시사점
train 밝기는 130~150 좁게 분포, test는 90~165로 넓게 분포	밝기/대비 증강 폭을 넓게 잡아야 함
train 배경은 100% 연회색·주백색, test는 배경 색조가 다양(보랏빛~밝은 청백색)	색상(hue)·채도 증강이 다소 필요
선명도가 촬영 각도(camera_la: 70/75/90)에 따라 이분 분포	회전(rotation)·기울임(shear) 증강 필요
알약에 각인 문자가 새겨져 있음	반전(flip) 시 문자가 거울상이 되어 실제와 달라질 위험 → 주의 필요
클래스 불균형(56개 클래스, 3~153개)	오버샘플링으로 이미 어느 정도 대응함 → 증강은 보조적 역할

2. 금지 사항 (성능을 떨어뜨릴 수 있는 요소로 판단)

기법	이유
강한 색상 반전/왜곡	알약 고유 색상이 클래스 구분의 핵심 특징 (배경/조명이 통제된 상태로 촬영됨)
강한 블러 추가	이미 촬영각도로 인한 자연스러운 블러 차이가 존재, 여기에 더 흐리게 하면 각인 인식을 방해
상하 반전(flipud)	알약 각인 문자가 거꾸로 보이면 실제 데이터와 다른 잘못된 패턴을 학습할 위험
과도한 회전(30도 이상)	알약은 원형·타원형·장방형이 섞여 있어 과도한 회전은 bbox 대비 비효율 발생

사용 모델

YOLO (Ultralytics 계열)

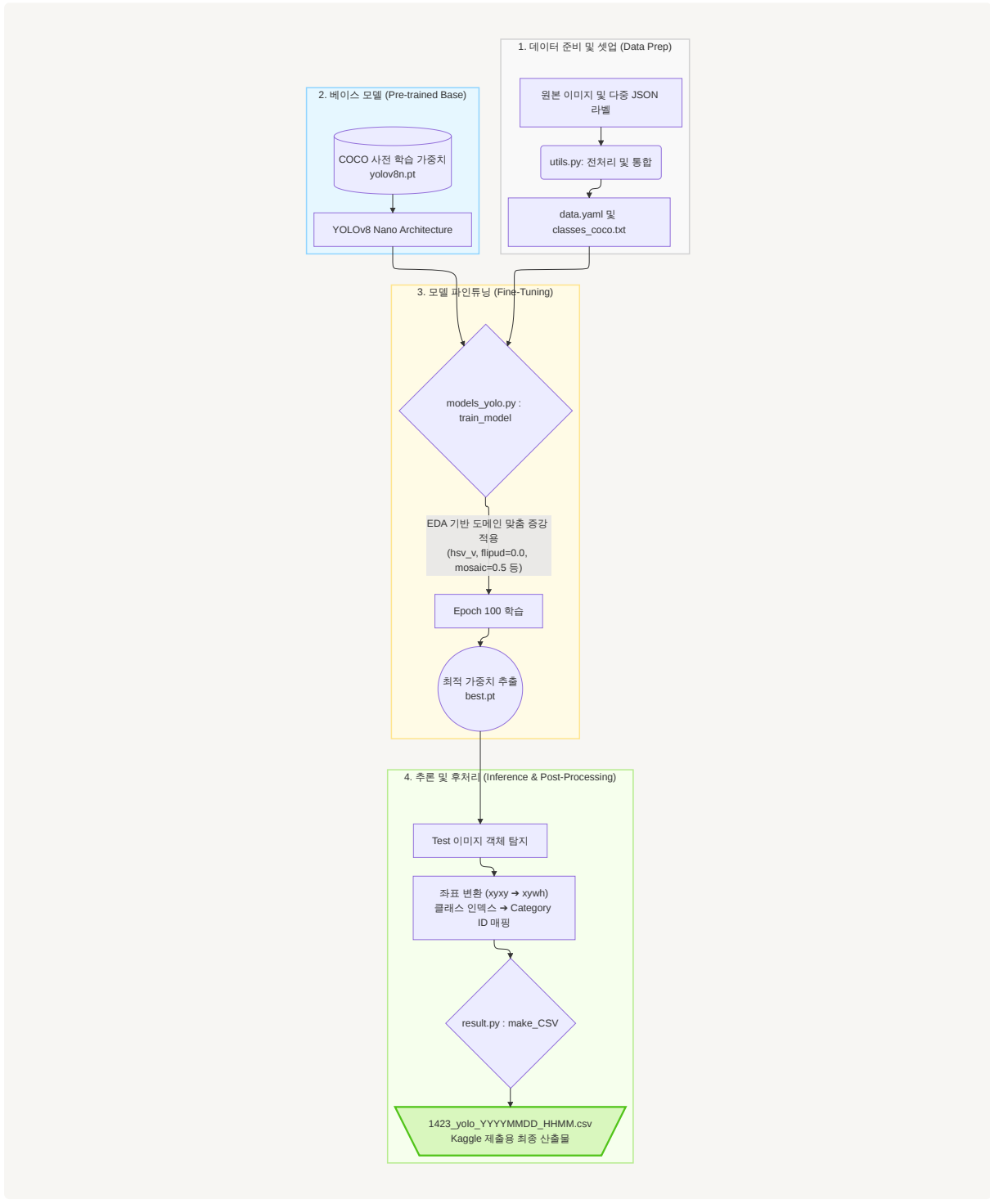
모델 개요

본 모델은 제한된 데이터와 실시간 처리라는 현장의 요구사항을 만족하기 위해, Ultralytics의 **YOLOv8 Nano**([yolov8n.pt](#)) 모델을 베이스라인으로 채택하여 전이 학습(Transfer Learning)을 수행했습니다. 구현된 코드는 모델의 뼈대를 그대로 가져오는 부분과, 약제 데이터 특성에 맞춰 세밀하게 조율된 파인튜닝(Fine-tuning) 및 후처리 부분으로 나뉘집니다.

- YOLOv8 Nano 아키텍처 및 사전학습 가중치 ([yolov8n.pt](#))
 - 이미지에서 기본적인 형태, 질감, 윤곽선 등의 범용적인 특징(Feature)을 추출하는 능력을 활용하기 위해 대규모 데이터셋(COCO)으로 사전 학습된 가중치를 그대로 가져왔습니다.
 - Backbone(특징 추출기)과 Neck(다양한 크기의 특징 융합) 구조를 처음부터 학습(Train from scratch)하지 않고 차용함으로써, 학습 시간을 단축하고 적은 알약 데이터셋(232장)으로도 과적합(Overfitting)을 방지할 수 있는 기반을 마련했습니다.
 - 파인튜닝 및 커스터마이징한 부분 (Fine-Tuning & Post-Processing)**
가져온 베이스 모델을 경구 약제 도메인에 맞게 아래와 같이 튜닝했습니다.

1. 의도적인 데이터 증강(Augmentation) 파라미터 튜닝

- 단순히 기본값을 사용하지 않고 EDA 인사이트를 반영하여 `train()` 함수의 하이퍼파라미터를 재설정했습니다.
- 적용:** 테스트 세트의 넓은 밝기/배경색 분포에 대응하기 위해 HSV 색공간 변경을 넉넉하게 부여(`hsv_v=0.5`, `hsv_s=0.6`)하고, 촬영 각도 보정을 위한 회전(`degrees=10`)을 추가했습니다.
- 제한:** 약제 표면의 각인 문자가 훼손되어 오인식되는 것을 막기 위해 상하 반전은 차단(`flipud=0.0`)하고, 여러 알약이 겹치는 과도한 합성 기법(`mosaic=0.5`)은 비중을 줄였습니다.



실험결과(Base)

파일명	1423_yolo_20260916_2149.csv
모델명	YOLO
버전	yolov8n
epoch	100
image size	640
learning rate	0.01(Default)
degrees(회전)	10
hsv_v(밝기)	0.5
hsv_h(색조)	0.02
hsv_s(채도)	0.6
fliplr(좌우반전)	0.3
flipud(상하반전)	0
translate(이동 변화)	0.1
scale(크기 변화)	0.3
mosaic	0.5
Leaderboard	0.3985

구분	항목명	최종 수치	설명 및 해석 포인트
1. 기본 정보 및 시간	epoch	100	전체 데이터셋을 100번 반복하여 학습 완료
	time	295.929 초	학습 소요 시간 (약 5분) - GPU
2. 손실 함수 (Loss) (낮을수록 좋음)	train/box_loss / val/box_loss	0.27524 / 0.26606	알약 위치(바운딩 박스) 예측 오차
	train/cls_loss / val/cls_loss	0.49882 / 0.54162	알약 종류(클래스) 분류 오차
	train/dfl_loss / val/dfl_loss	0.81479 / 0.79283	박스 경계선을 정교하게 맞추기 위한 손실 (DFL)
	해석 요약	-	훈련/검증 손실이 비슷하게 수렴하여 과적합 없이 일반화가 잘 됨
3. 성능 평가 지표 (1에 가까울수록 좋음)	metrics/precision(B)	0.94823	정밀도 약 94.8% (오탐지가 적음)
	metrics/recall(B)	0.99500	재현율 99.5% (미탐지가 거의 없음)
	metrics/mAP50(B)	0.98409	IoU 0.5 기준 평균 정밀도 약 98.4%
	metrics/mAP50-95(B)	0.94838	IoU 0.5~0.95 기준 평균 정밀도 약 94.8% (위치와 경계선을 매우 정밀하게 검출)
4. 학습률 (Learning Rate)	lr/pg0 , lr/pg1 , lr/pg2	0.0000033233	옵티마이저 파라미터 그룹별 최종 학습률 (초기 대비 대폭 감소하여 안정적인 미세 조정 완료)

성능/정확도 평가

[Yolo] 정성적 평가

모델 학습할 때 사용하는 데이터 로더(Data Loader)의 모자이크(Mosaic) 및 증강(Augmentation) 결과



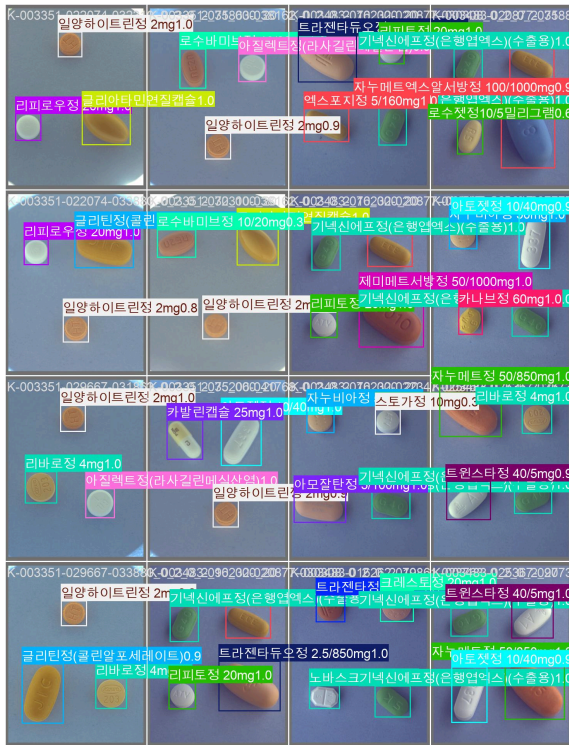
- **모자이크 증강 (Mosaic Augmentation):** YOLO는 학습 효율을 높이기 위해 4장의 서로 다른 알약 이미지를 하나의 큰 격자로 이어 붙여서(모자이크) 모델에 입력합니다.
- **다양한 배치(Batch):** `batch0` 과 `batch1802` 는 학습 진행 과정에서 서로 다른 시점(또는 다른 미니배치 번호)에 무작위로 샘플링되어 들어간 데이터 묶음입니다.



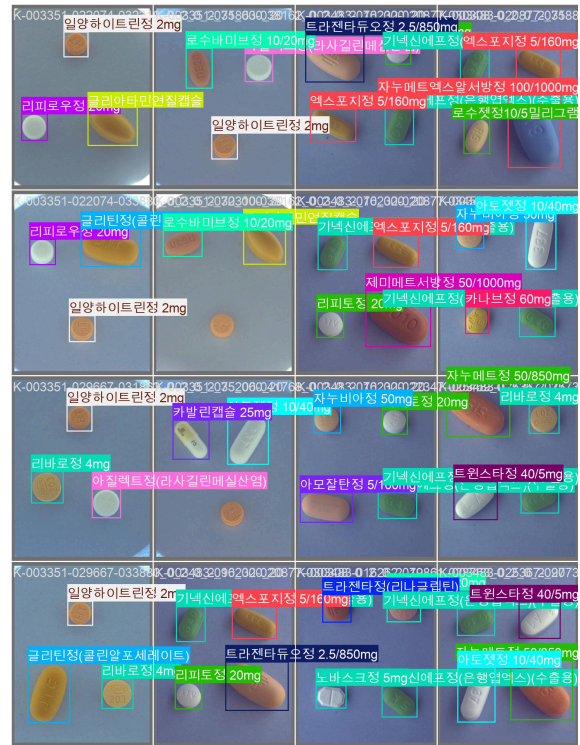
val_batch0_labels



val_batch0_pred



val_batch1_labels



val_batch1_pred

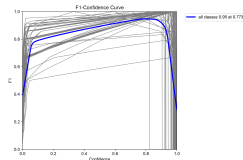


- **정답과 예측의 높은 일치도:** 라벨과 실제 예측한 이미지 모델의 추론 결과를 보여줍니다. 두 그룹을 비교했을 때, 누락되거나 다른 약으로 오분류된 케이스 없이 정답과 예측이 거의 완벽하게 일치합니다.
- **높은 신뢰도(Confidence Score):** 예측 이미지의 바운딩 박스 상단 텍스트를 보면 클래스명 옆에 '1.0', '0.9' 등의 숫자가 적혀 있습니다. 이는 모델이 해당 알약을 판별할 때 90%~100%의 매우 강한 확신을 가지고 정답을 맞혔음을 의미합니다.
- **정교한 위치 추정(Localization):** 모델이 예측한 바운딩 박스가 알약의 실제 경계선에 빈틈없이 타이트하게(Tight) 그려져 있습니다. 객체의 일부가 잘려 나가거나 배경이 너무 넓게 포함되지 않고, 알약의 영역만 정확하게 찾아냈습니다.
- **다중 객체 탐지 성능:** 하나의 이미지 안에 '일양하이트린정 2mg'처럼 동일한 알약이 여러 개 흩어져 있거나, 크기와 모양이 다른 여러 종류의 알약이 섞여 있는 환경에서도 모델은 단 하나의 알약도 놓치지 않고 개별적으로 정확하게 검출해냈습니다.

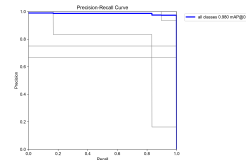
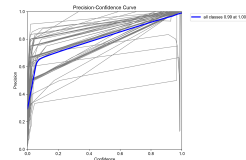
[Yolo] 정량적 평가

전반적인 모델 성능 요약

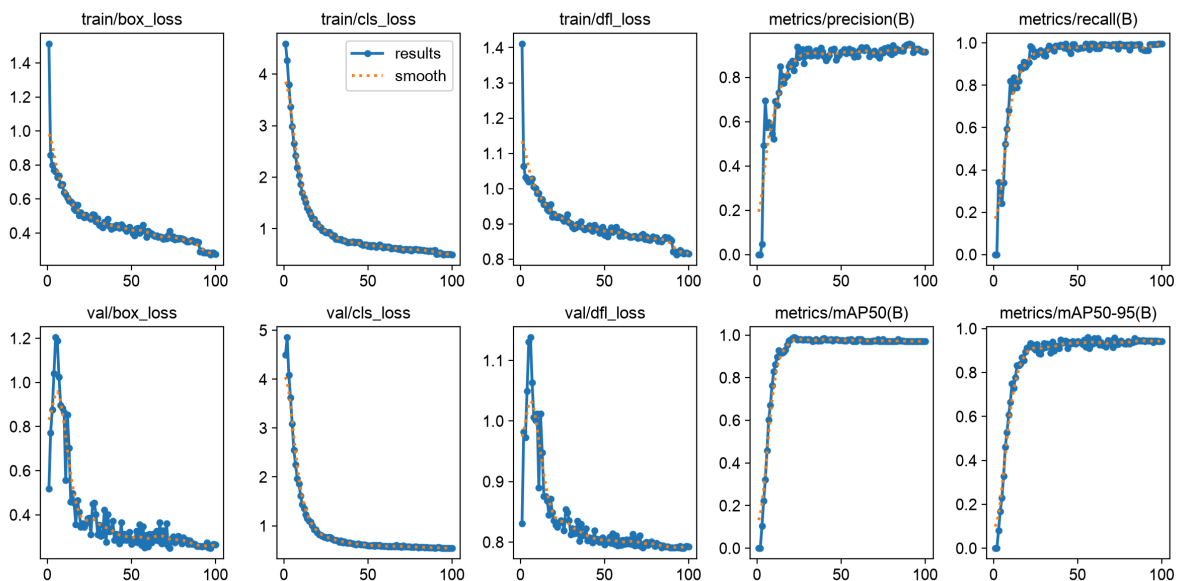
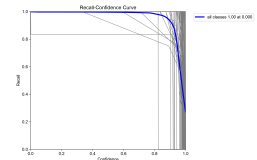
모델은 100 Epoch 동안 안정적으로 학습되었으며, 객체 검출(Object Detection) 및 분류(Classification)에서 매우 우수한 성능을 달성했습니다.



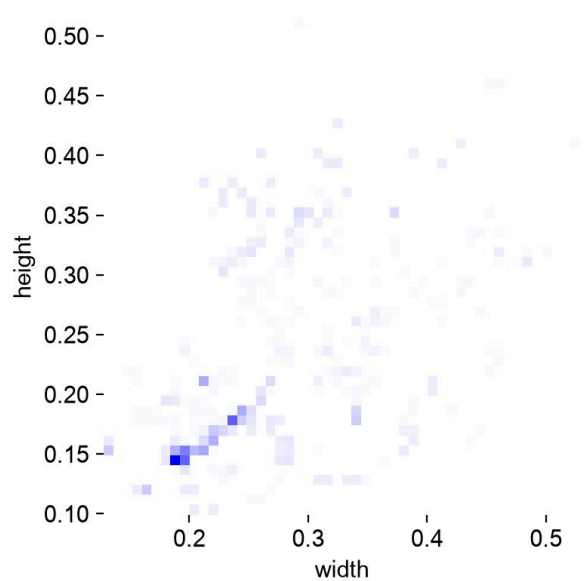
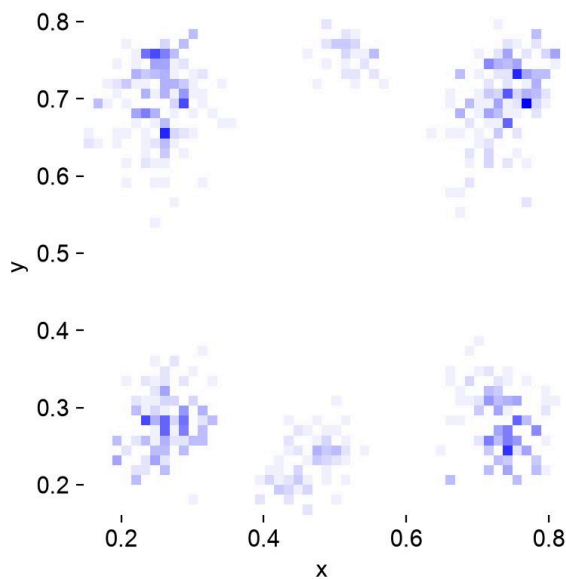
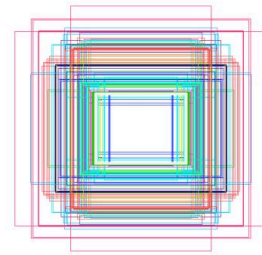
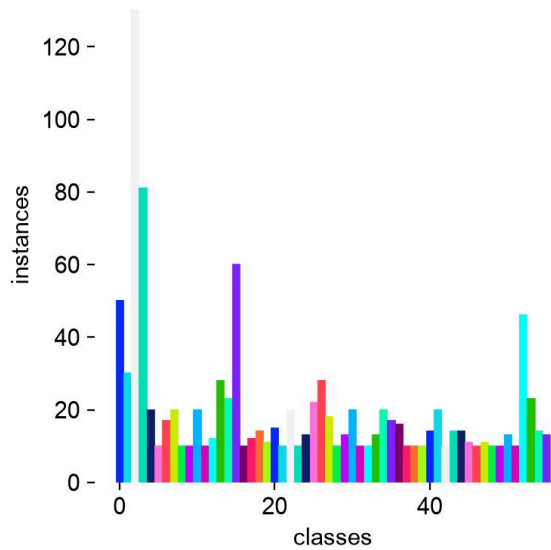
최적 F1-Score: 0.9



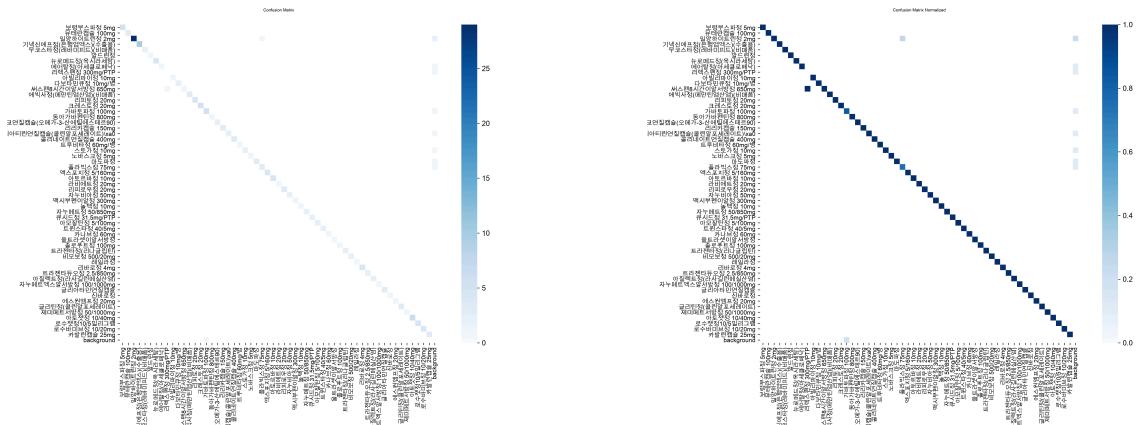
mAP@0.5 (평균 정밀도):
0.980



- **손실(Loss) 감소 추이:** `train/box_loss`, `train/cls_loss`, `train/dfl_loss` 모두 100 Epoch 동안 지속적으로 하락하며 안정화되었습니다. Validation Loss 역시 Training Loss와 유사한 궤적으로 감소하여 **과적합 (Overfitting) 현상 없이 훌륭하게 일반화**되었습니다.
- **성능 지표 수렴:** Precision, Recall, mAP50, mAP50-95 지표 모두 약 20~30 Epoch 부근에서 급격히 상승한 후 높은 수치(0.9 이상)를 유지하며 수렴했습니다. 이는 데이터셋이 모델이 학습하기에 충분히 명확한 특징을 가지고 있음을 시사합니다.



- **클래스 불균형 (Class Imbalance):** 상단 좌측 막대그래프를 보면 클래스별 인스턴스(객체) 수에 차이가 있습니다. 어떤 클래스는 80개 이상의 데이터가 있지만, 대부분의 클래스는 10~20개 수준입니다. 그럼에도 불구하고 혼동 행렬에서 성능 저하가 크지 않은 것으로 보아 모델이 특징을 잘 추출해냈습니다.
- **바운딩 박스 위치 (x, y):** 객체의 중심점(x, y)이 주로 이미지 중앙(0.5, 0.5)보다는 특정 사분면에 군집화되어 있거나, 넓게 퍼져있는 형태(하단 좌측 산점도)를 보입니다.
- **바운딩 박스 크기 (width, height):** 객체의 너비와 높이가 주로 이미지 전체 크기의 10%~20% (0.1~0.2) 부근에 집중되어 있습니다(하단 우측 산점도). 이는 촬영된 알약의 크기와 배율이 대체로 일정했음을 보여줍니다.

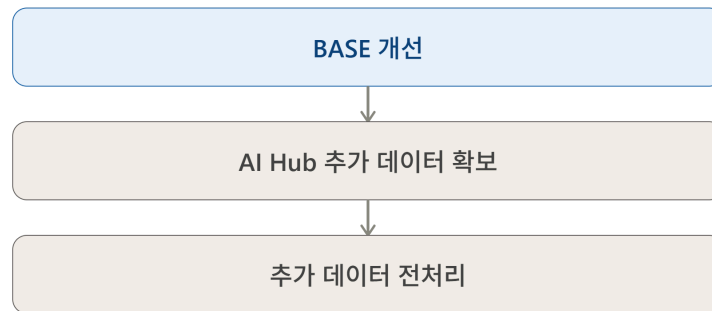


- 정규화된 혼동 행렬(`confusion_matrix_normalized_2.jpg`)을 보면, 짙은 파란색 선이 대각선을 따라 완벽하게 일직선으로 형성되어 있습니다.
- 이는 모델이 다양한 종류의 알약(약 40여 종 이상)을 서로 헷갈리지 않고 95% 이상의 정확도로 완벽하게 구분해내고 있다는 뜻입니다. 백그라운드(배경)를 알약으로 오인하거나, 알약을 배경으로 놓치는 비율(마지막 행/열)도 극히 미미합니다.

결론 :

현재 모델은 주어진 데이터셋에 대해 성능을 잘 발휘하고 있습니다. 파라미터 설정은 고정해도 될 것으로 보여 집니다.

실험결과(Base 개선)

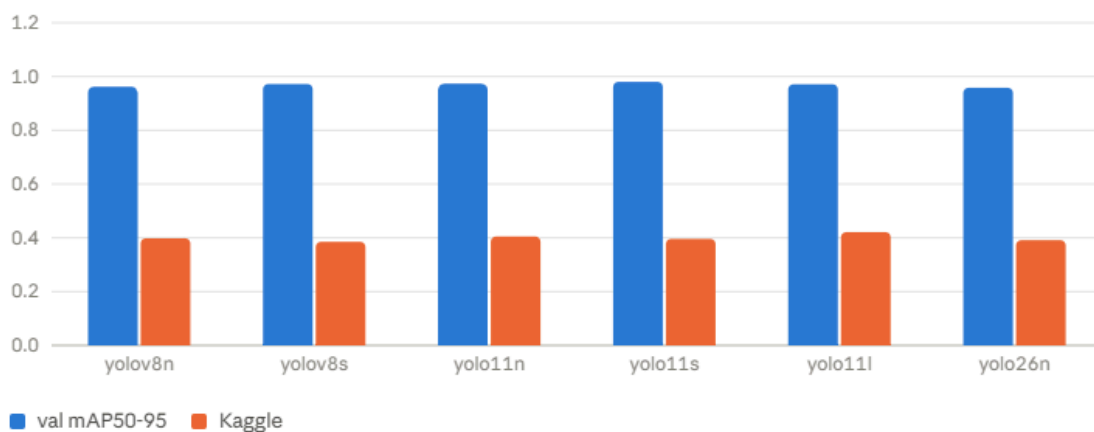


위 정략적 평가를 확인 했을 때 차트를 통해 안정적이라는 것을 확인 할 수 있었습니다. 이는 EDA 결과를 바탕으로 설정한 초기 파라미터가 우연히 현재 데이터셋에 맞았던 것 같습니다. 그래서 현재 설정한 파라미터 값을 고정하고 모델 버전 및 크기에 따른 Kaggle Score 값의 변화를 확인해보기로 했습니다.

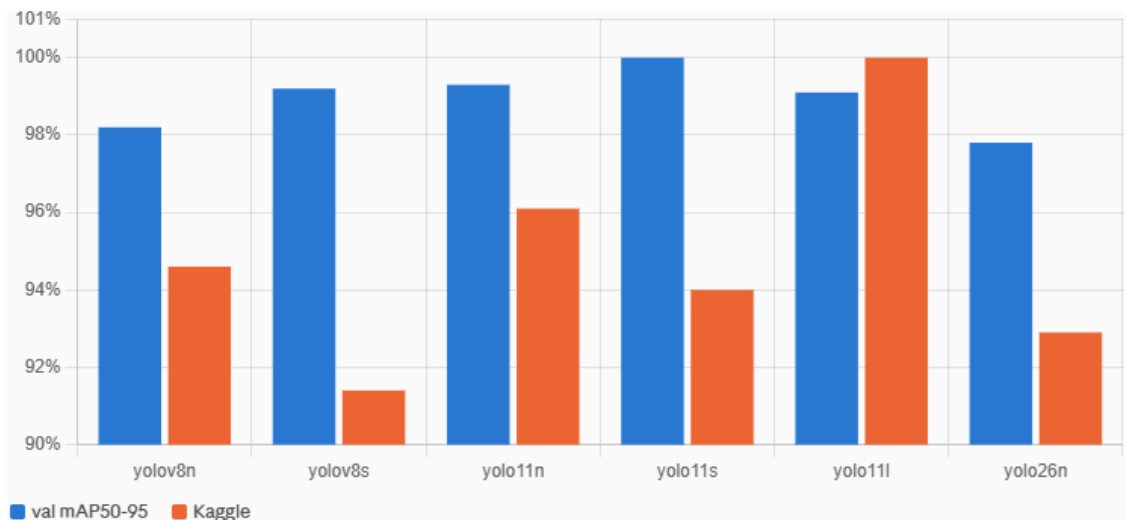
추가로 hsv_h(색조), Scale, mosaic 등 값을 변경해서 실험을 해봤으나 정확한 계산으로 변경한게 아닌 감으로 값을 변경하여 실험 하다보니 Score 값은 오히려 감소 했습니다. 이런식의 실험은 의미가 없다고 판단 했습니다. 그래서 안정적인 현재 값을 고정하여 실험을 진행 했습니다.

모델	val mAP50-95	Kaggle
yolov8n	0.962	0.3985
yolov8s	0.972	0.3853
yolo11n	0.973	0.40507
yolo11s	0.98	0.39609
yolo11l	0.971	0.42134
yolo26n	0.958	0.39142

모델별 성능 비교



차이를 더 확실히 확인하기 위해 최고값을 100%로 보고 상대 비교를 했습니다



- val mAP 점수만 보면 실험이 거의 구분이 안 될 정도로 비슷한데 Kaggle Score는 차이가 큰 것으로 보입니다
- val image(51장)은 train과 조건이 비슷해서 어떤 모델을 쓰든 mAP가 거의 항상 0.96~0.98 근처로 나오고 있습니다 이는 모델 간 실직적 차이를 구별할 능력이 없다고 판단 됩니다

1. YOLO11의 성능이 높았던 이유

YOLO11에는 YOLOv8에 없던 **C2PSA 블록**(어텐션(attention) 기반 모듈)이 추가되었습니다.

- **어텐션(attention)** : 이미지에서 어느 영역에 더 집중할지를 모델이 스스로 판단하는 기능입니다. 알약 이미지의 경우 배경은 덜 보고, 각인 문자가 새겨진 영역에 더 집중하도록 돕습니다.
- **모델 크기에 따른 차이** : n(nano) 모델은 엣지 디바이스용 경량 모델이라 어텐션 구조가 있어도 이를 충분히 활용할 용량이 부족합니다. 반면 l(large) 모델은 층이 더 깊고 채널이 넓어, 어텐션이 각인 문자처럼 세밀한 특징에 집중하도록 더 정교하게 학습될 수 있습니다.
- 결과적으로 어텐션 구조와 큰 모델의 표현력이 함께 작용하여 성능 향상으로 이어진 것으로 판단됩니다.

2. s 모델의 성능이 상대적으로 낮았던 이유

n 모델은 파라미터가 적어 과적합(overfitting) 위험이 낮고, l 모델은 표현력이 커서 각인 문자와 같은 미세한 차이를 학습할 여력이 있습니다. s 모델은 두 장점을 모두 충분히 얻지 못한 중간 지점이었을 가능성이 있습니다. 다만 이는 1회 실험 결과에 근거한 추정이므로, 학습 시 무작위성(시드(seed) 차이)에 의한 편차일 가능성도 함께 고려해야 합니다.

3. YOLO26의 성능이 낮았던 이유

① 주요 개선 사항이 본 과제의 특성과 맞지 않았을 가능성

YOLO26의 소형 객체 인식 개선은 항공·드론 사진처럼 화면에서 차지하는 비율이 매우 작은 객체에 초점을 둔 기법입니다. 본 과제의 알약은 이미지 내 비중이 비교적 크기 때문에 해당 개선의 효과를 크게 얻지 못했을 수 있습니다.

② 소규모 데이터셋과 최신 학습 기법 간의 불일치 가능성

새로운 옵티마이저(optimizer)와 손실 함수(loss function)는 주로 대규모 데이터셋에서 검증된 기법으로, 본 과제처럼 데이터가 적은 환경에서는 충분히 최적화되지 않았을 수 있습니다.

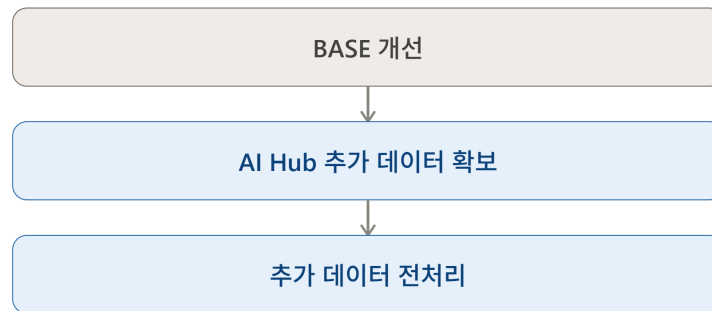
4. 결론

테스트 데이터(842장)는 밝기, 촬영 각도, 신규 클래스 등 다양한 조건이 혼재되어 있어, 모델의 일반화 성능, 특히 어텐션 구조와 미세한 각인 구분 능력의 차이가 점수에 반영된 것으로 보입니다.

또한 이번 실험은 **최신 버전의 모델이 항상 더 좋은 성능을 보장하지 않으며, 실제 데이터에 적합한지 검증한 후 선택해야 한다**는 점을 보여줍니다.

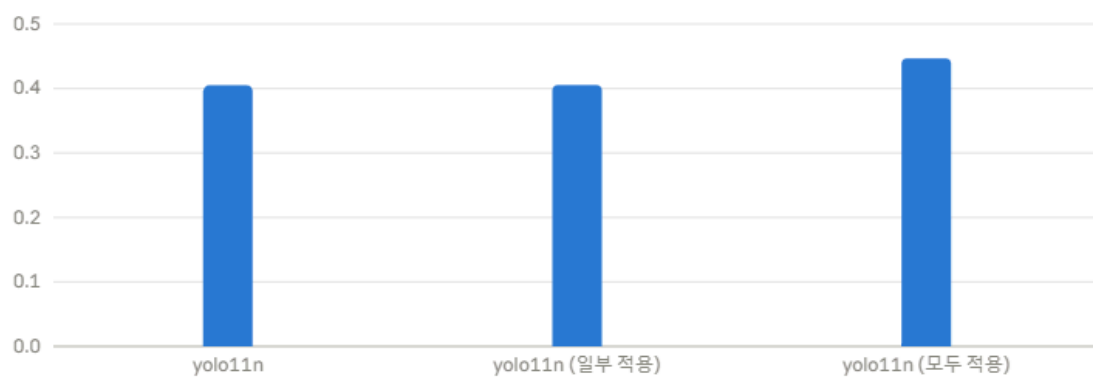
기존 제공 데이터 기준으로는 **YOLO11 (Score 0.42134)** 을 최종 모델로 선정하였으며, 이후 AI Hub 데이터를 활용해 클래스를 추가 확보한 뒤 다음 실험을 진행하였습니다.

실험결과(AI HUB 데이터 확보)



먼저 모델을 yolo11n으로 고정하고 AI HUB 데이터 일부 적용, 모두 적용 두 가지로 나눠 확인 했습니다

Kaggle 점수

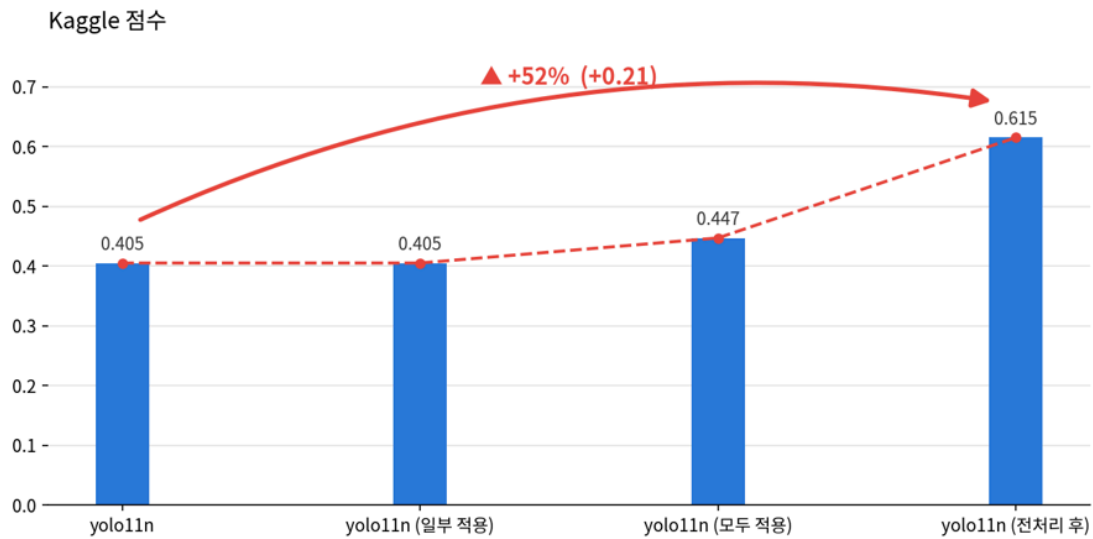


모델	Kaggle
yolo11n	0.40507
yolo11n(일부 적용)	0.40534
yolo11n(모두 적용)	0.44664

일부 데이터만 적용 했는데 yolo11n을 사용 했을 때는 미비한 증가가 있었고, TL_1, TL_3~8의 데이터를 모두 적용 했을 때는 보다 큰 증가가 있었습니다. 이는 클래스 확보로 기존 데이터로는 추론하지 못했던 알약을 추론하면서 증가 했다고 보여 집니다.

AI HUB 데이터 확보로 train image와 클래스가 확보 되었음에도 불구하고 이전과 크게 증가 되지 않았습니다. 여기서 놓친 부분은 클래스 확보에만 집중했지 전처리를 진행하지 않았습니다. 아래 두 가지 사항을 처리 했습니다.

- bbox 이상치 제거
- 확보 한 데이터에서 category_id를 dl_idx를 사용했었는데 drug_N과 1차이가 발생함을 확인 했습니다. 기존 제공 된 데이터를 확인해보니 drug_N을 사용하는게 맞다고 판단 했습니다.

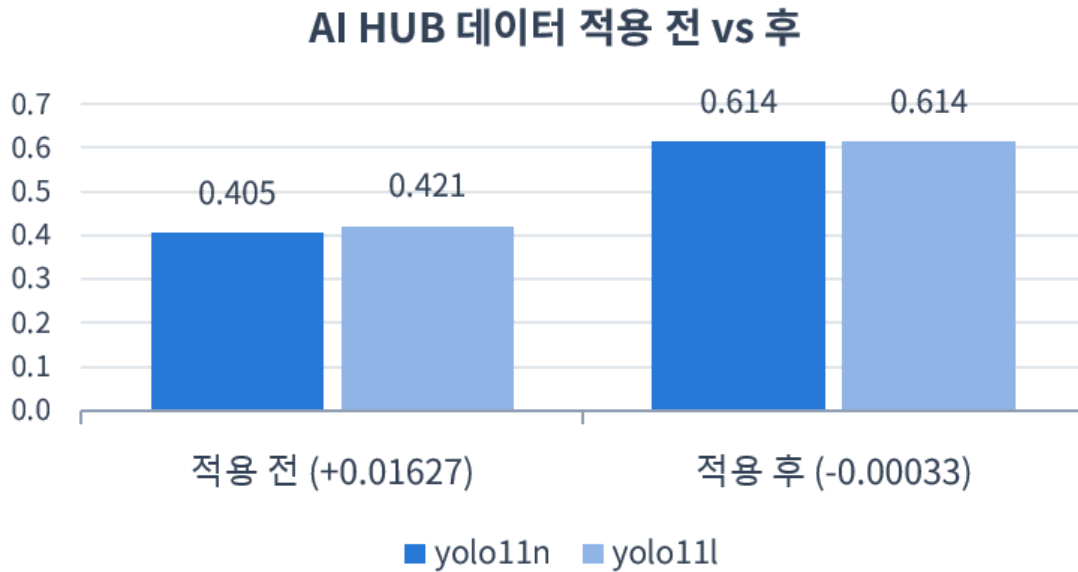


모델	Kaggle
yolo11n	0.40507
yolo11n (일부 적용)	0.40534
yolo11n (모두 적용)	0.44664
yolo11n (전처리 후)	0.61383

bbbox 이상치를 제거하고 drug_N을 사용하는게 맞았습니다. 확보한 데이터로 학습은 잘 되었지만 category_id가 틀려서 Score가 오르지 않는 문제였습니다.

이를 통해 확실해졌습니다. 모델의 성능을 가장 크게 끌어올리는 것은 파라미터 설정, 모델의 버전이나 크기가 아닌 EDA를 통해 전처리 된 깔끔한 데이터 입니다. 데이터의 중요성을 다시 한번 크게 느낀 실험이었습니다

실험결과(yolo11n vs yolo11l)



모델	Kaggle	차이 (n 기준)
yolo11n	0.61383	-
yolo11l	0.61350	-0.00033

AI HUB 데이터 적용전에는 yolo11n (0.40507), yolo11l (0.42134) 사이에 확실한 차이가 있었는데, 적용 후에는 거의 같아졌습니다. 데이터가 부족할 때는 모델 크기가 변별력을 가졌지만 데이터가 확보 되면서 모델 크기 차이가 무의미해진 것으로 보입니다.

- 적용 전 : 클래스가 56개 뿐이고 정보가 제한적이니 더 큰 모델이 그 안에서 조금이라도 더 정교하게 학습할 여지가 있어서 모델 크기가 변별력을 가졌습니다.
- 적용 후 : 데이터 확보로 이미지, 클래스가 충분히 확보가 되니 작은 모델(n)도 이미 충분히 학습할 수 있는 상태가 되면서 이 지점부터는 모델 크기를 늘려도 추가로 얻어낼 정보가 거의 없었던 것으로 보입니다. 작은 모델도 충분히 일반화 할 수 있는 양으로 큰 모델의 이점이 작용하지 않게 되었을 수도 있을 것 같습니다
- 데이터가 충분해지면 모델 성능이 수렴
- 현재 데이터, 전처리 수준으로는 현재 Score가 한계 일 수 있습니다.
- 현재 상태에서는 모델을 키우는 것보다 가볍고 빠른 yolo11n을 사용하는 것이 효율적 입니다.

실험결과(yolo vs Faster R-CNN)

Faster R-CNN Kaggle Score : 0.24205 (AI HUB 데이터 미적용)

초기에는 동일한 조건에서 어떤 모델이 더 나은 성능을 보이는가를 실험 목표로 설정 했으나 이 설정 자체가 적절하지 않았습니다.

1. 모델 구조가 근본적으로 다릅니다

- 구조가 다르기 때문에 각 모델에 적합한 입력 크기, 학습률, 데이터 증강, 학습 에폭 수 등도 서로 다릅니다

2. 완전히 동일한 조건을 만드는 것이 불가능 합니다

- 사용한 프레임워크(Ultralytics) 마다 기본 설정, 데이터 증강 파이프라인, 손실 함수, 옵티마이저 구성이 다릅니다
- 하이퍼파라미터를 억지로 맞추면 오히려 한쪽 모델에 불리한 조건이 되어 공정한 비교가 되지 않습니다

3. 올바른 비교 방향

- 동일 조건 비교가 아니라 각 모델을 충분히 튜닝 했을 때의 최종 성능을 비교하는 것이 더 타당한 접근입니다

결론

1. 모델 버전·크기

- 크고 최신인 모델이 **항상 좋지는 않음**

2. AI HUB 데이터·전처리

- 전처리 후 0.405 → 0.613, **가장 큰 성능 향상**

3. yolo11 모델 크기

- 데이터 확보 후 n:1 성능이 **수렴** (0.61383 vs 0.61350)

4. Faster R-CNN 비교

- 동일 조건 비교는 부적절 → **모델별 튜닝 후 비교**가 타당

성능을 결정한 핵심은 모델보다 **데이터 품질(전처리)**이며 최종 선택은 11과 동등하고 더 가볍고 빠른 **yolo11n (0.61383)** 을 사용하는 것이 효율적이다